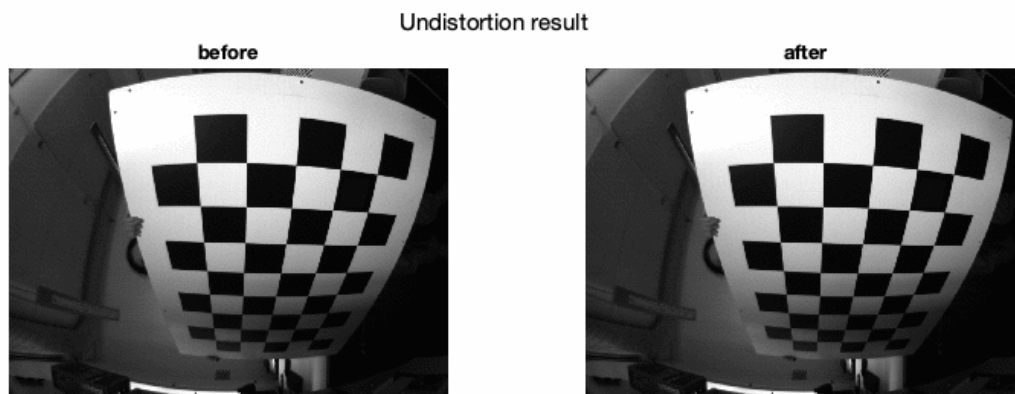


Calibrating Cameras with Zhang's Method

📅 July 27, 2025 • 💬 0 • ❤️ 5

Camera calibration is a fundamental step in many computer vision applications, from 3D reconstruction to augmented reality. It involves estimating the intrinsic parameters of a camera (like focal length, principal point, and lens distortion) and its extrinsic parameters (rotation and translation relative to a world coordinate system). Among the various calibration techniques, Zhang's method stands out for its simplicity and effectiveness, requiring only a planar checkerboard pattern observed from different viewpoints.



Understanding the Theory Behind Zhang's Method

Zhang's method leverages the concept of homography, which is a 2D planar projective transformation. When a planar object (like a checkerboard) is viewed by a camera, the relationship between the 3D points on the plane and their 2D projections in the image can be described by a homography matrix. Let $\mathbf{p}_w = [x, y, z]^T$ be a 3D point on the checkerboard plane in world coordinates, and $\mathbf{p}_i = [u, v]^T$ be its corresponding 2D image point. The relationship between $\mathbf{p}_w$ and $\mathbf{p}_i$ can be expressed as:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K[R|\mathbf{t}] \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where s is an arbitrary scale factor, K is the intrinsic camera matrix, R is the rotation matrix, and $\mathbf{t}$ is the translation vector. The intrinsic camera matrix K is typically defined as:

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Here, f_x and f_y are the focal lengths in pixels along the x and y axes, (c_x, c_y) is the principal point (the intersection of the optical axis with the image plane), and γ is the skew parameter (often assumed to be zero for modern cameras).

To avoid a rank deficiency problem and uniquely calculate the camera intrinsic and extrinsic parameters from a single view, Zhang's method assumes $z = 0$ for the world coordinates of the checkerboard plane. We can simplify the projection equation for points on the $z = 0$ plane:

$$\begin{aligned} s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} &= K[\mathbf{r}_1|\mathbf{r}_2|\mathbf{r}_3|\mathbf{t}] \begin{bmatrix} x \\ y \\ 0 \\ 1 \end{bmatrix} \\ &= K[\mathbf{r}_1|\mathbf{r}_2|\mathbf{t}] \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \\ &= H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \end{aligned}$$

where $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ are the column vectors of the rotation matrix R . The matrix $H = K[\mathbf{r}_1|\mathbf{r}_2|\mathbf{t}]$ is the homography matrix, which is a 3×3 matrix. This homography relates the 2D world coordinates on the plane (x, y) to the 2D image coordinates (u, v) .

To construct a linear equation, let

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

Then, we expand the above equation:

$$\begin{aligned} su &= xh_{11} + yh_{12} + h_{13} \\ sv &= xh_{21} + yh_{22} + h_{23} \\ s &= xh_{31} + yh_{32} + h_{33} \end{aligned}$$

By eliminating the scale factor, s , and using direct linear transformation (DLT), a linear equation $A\mathbf{h} = 0$ is obtained, where

$$A = \begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & xu & yu & u \\ 0 & 0 & 0 & -x & -y & -1 & xv & yv & v \end{bmatrix}$$

and $\mathbf{h} = [h_{11}, h_{12}, \dots, h_{33}]^T$.

The crucial insight of Zhang's method comes from the properties of the rotation matrix. Since $\mathbf{r}_1$ and $\mathbf{r}_2$ are orthogonal and unit vectors, they satisfy the following conditions:

$$\begin{aligned} \mathbf{r}_1^T \mathbf{r}_2 &= 0 \\ \|\mathbf{r}_1\|^2 &= 1 \\ \|\mathbf{r}_2\|^2 &= 1 \end{aligned}$$

Substituting $\mathbf{r}_1 = K^{-1}\mathbf{h}_1$ and $\mathbf{r}_2 = K^{-1}\mathbf{h}_2$ (where $\mathbf{h}_1$ and $\mathbf{h}_2$ are the first two columns of the homography matrix $H = [\mathbf{h}_1 | \mathbf{h}_2 | \mathbf{h}_3]$), we get:

$$\begin{aligned} \mathbf{h}_1^T K^{-T} K^{-1} \mathbf{h}_2 &= 0 \\ \mathbf{h}_1^T K^{-T} K^{-1} \mathbf{h}_1 &= \mathbf{h}_2^T K^{-T} K^{-1} \mathbf{h}_2 = 1 \end{aligned} \tag{1}$$

Let $B = K^{-T} K^{-1}$. This matrix B is symmetric and positive definite. Therefore, there are six unknown parameters in B . The two equations above provide two constraints on the intrinsic parameters for each observed image of the checkerboard. If we capture images from at least three different orientations, we can set up a system of linear equations to solve for the elements of B . Once B is determined, the intrinsic matrix K can be found through Cholesky decomposition.

After obtaining K , the extrinsic parameters (R and $\mathbf{t}$) for each view can be calculated using the columns of the homography matrix:

$$\begin{aligned}\mathbf{r}_1 &= \lambda K^{-1} \mathbf{h}_1 \\ \mathbf{r}_2 &= \lambda K^{-1} \mathbf{h}_2 \\ \mathbf{r}_3 &= \mathbf{r}_1 \times \mathbf{r}_2 \\ \mathbf{t} &= \lambda K^{-1} \mathbf{h}_3\end{aligned}$$

where $\lambda = 1/||K^{-1}\mathbf{h}_1||$ is a scaling factor to ensure $\mathbf{r}_1$ and $\mathbf{r}_2$ are unit vectors.

Finally, to refine the estimated parameters and account for lens distortion (radial and tangential), a non-linear optimization (e.g., Levenberg-Marquardt) is performed. This optimization minimizes the reprojection error, which is the sum of the squared distances between the observed image points and the points projected onto the image plane using the current camera parameters.

MATLAB Implementation

The provided MATLAB code demonstrates a practical implementation of Zhang's method. Let's walk through the key functions:

Detect checkerboard's points

The function, `detectCheckerboardPoints`, a part of MATLAB's Computer Vision Toolbox, is used to automatically detect the checkerboard corners in a set of input images. It returns the pixel coordinates of these corners, `imagePoints`, and the size of the checkerboard grid, `boardSize`.

Compute homography

The function, `computeHomography`, calculates the homography matrix $\mathbf{H}$ that maps 3D world points ($\mathbf{P}_w$) on the checkerboard plane to 2D image points ($\mathbf{P}_i$). It constructs a system of linear equations $\mathbf{A} \cdot \mathbf{h} = \mathbf{0}$, where $\mathbf{h}$ is a vector containing the elements of the homography matrix, and then solves for $\mathbf{h}$ using Singular Value Decomposition (SVD).

```

function H = computeHomography(P_w, P_i)
    numPoints = size(P_w,1);
    A = zeros(2*numPoints, 9);

    for i = 1:numPoints
        X = P_w(i,1);
        Y = P_w(i,2);
        u = P_i(i,1);
        v = P_i(i,2);

        A(2*i-1,:) = [-X, -Y, -1, 0, 0, 0, X*u, Y*u, u];
        A(2*i, :) = [ 0, 0, 0, -X, -Y, -1, X*v, Y*v, v];
    end

    [~,~,V] = svd(A);
    H = reshape(V(:,end), [3,3])'; % the smallest singular value
end

```

Compute intrinsic parameter

This section of the main script sets up the system of linear equations based on the homography matrices to solve for the matrix B .

In Eq. (1), we've set $B = K^{-T}K^{-1}$. Since the matrix B is symmetric and positive definite, the number of independent elements in B is six, thus letting

$$B = \begin{bmatrix} b_1 & b_2 & b_4 \\ b_2 & b_3 & b_5 \\ b_4 & b_5 & b_6 \end{bmatrix}.$$

Using $\mathbf{h}_1 = [h_{11}, h_{21}, h_{31}]^T$ and $\mathbf{h}_2 = [h_{12}, h_{22}, h_{32}]^T$, we can expand the constraints in Eq. (1) in terms of b_* :

$$\begin{aligned}
& \mathbf{h}_i^T B \mathbf{h}_j \\
&= (h_{1i} h_{1j}) b_1 \\
&+ (h_{1i} h_{2j} + h_{2i} h_{1j}) b_2 \\
&+ (h_{2i} h_{2j}) b_3 \\
&+ (h_{3i} h_{1j} + h_{1i} h_{3j}) b_4 \\
&+ (h_{2i} h_{3j} + h_{3i} h_{2j}) b_5 \\
&+ (h_{3i} h_{3j}) b_6,
\end{aligned}$$

where $i, j \in \{1, 2\}$. Then, for each homography matrix, two constraints should be met, thus satisfying $V\mathbf{b} = \mathbf{0}$ for a total of N homographies, where $V \in \mathbb{R}^{2N \times 6}$ and $\mathbf{b} = [b_1, b_2, \dots, b_6]^T$ is the null space of V .

The `v()` anonymous function helps in constructing these equations. Once $\mathbf{b}$ (the vectorized form of B) is obtained via SVD, B is reconstructed, and then K (the intrinsic matrix) is derived using Cholesky decomposition.

```

% b11 b12 b22 b13 b23 b33
v = @(H,i,j) [
    H(1,i)*H(1,j), ...
    H(1,i)*H(2,j) + H(2,i)*H(1,j), ...
    H(2,i)*H(2,j), ...
    H(3,i)*H(1,j) + H(1,i)*H(3,j), ...
    H(3,i)*H(2,j) + H(2,i)*H(3,j), ...
    H(3,i)*H(3,j)
];

for i = 1:numImages
    H{i} = computeHomography(objectPoints, imagePoints(:, :, i));

    v1 = v(H{i}, 1, 2);
    v2 = v(H{i}, 1, 1) - v(H{i}, 2, 2);

    V = [V; v1; v2];
end

[~,~,Vmat] = svd(V);
b = Vmat(:,end);
B = [b(1), b(2), b(4); b(2), b(3), b(5); b(4), b(5), b(6)];
if det(B) < 0
    B = -B;
end
Kinv = chol(B,'upper');
K = inv(Kinv/Kinv(3,3));

```

Compute extrinsic parameter

With the initial intrinsic matrix K , the rotation matrices R and translation vectors t for each view are calculated from the homography matrices.

```
R = {};  
t = {};  
for i = 1:numImages  
    r1 = K\H{i}(:,1);  
    r2 = K\H{i}(:,2);  
  
    n = norm(r1);  
    r1 = r1 / norm(r1);  
    r2 = r2 / norm(r2);  
  
    r3 = cross(r1, r2);  
    r3 = r3 / norm(r3);  
  
    R{i} = [r1 r2 r3];  
    t{i} = K\H{i}(:,3);  
    t{i} = t{i} / n;  
  
    if t{i}(3) < 0 % the plane lies behind the camera  
        t{i} = -t{i};  
        R{i} = -R{i};  
    end  
end
```

Compute reprojection error

This function is the core of the non-linear optimization. It takes the current camera parameters (intrinsic and extrinsic), projects the 3D `objectPoints` onto the image plane, and then calculates the difference between these projected points and the observed `imagePoints`. This difference is the reprojection error that the optimization algorithm tries to minimize.

```

function err = reprojectionError(params, objectPoints, imagePoints)
    [fx, fy, s, cx, cy, distCoeffs, R, t] = parse_params(params);

    K = [fx, s, cx; 0, fy, cy; 0, 0, 1];

    err = [];
    numPoints = size(objectPoints,1);
    numImages = (length(params)-10)/7;

    for i = 1:numImages
        imagePoint = imagePoints(:,i);
        X = [R{i} t{i}; 0 0 0 1] * [objectPoints,
ones(size(objectPoints,1),1)]';
        projected = X(1:3,:) ./ X(3,:);
        projected = K * distort(projected, distCoeffs);
        projected = projected(1:2,:)';

        % compute reprojection error
        err = [err; imagePoint(:) - projected(:)];
    end
end

```

Refine intrinsic and extrinsic parameters, and compute distortion parameters

These functions utilize the non-linear optimization using `lsqnonlin` (Levenberg-Marquardt algorithm). They take the initial parameters and iteratively refine them by minimizing the reprojection error. The `variations` parameter controls the bounds for the parameter search during optimization, allowing for a staged refinement process.

```
function params_opt = myEstimateCameraParameters(K, R, t, objectPoints,
imagePoints)

    params_init = [K(1,1), K(2,2), K(1,2), K(1,3), K(2,3), zeros(1,5)]; %
fx, fy, s, cx, cy, distCoeffs
    for i = 1:length(R)
        quat = rotm2quat(R{i});
        params_init = [params_init, quat, t{i}'];
    end

    options = optimoptions('lsqnonlin', 'Display', 'final-detailed',
'FunctionTolerance', 1e-10, 'StepTolerance', 1e-20, 'MaxIterations', 2000,
'Algorithm', 'levenberg-marquardt');

    variations = [0.2, 0.2, 0.2, 0.2, 0.2, Inf, Inf, 0, 0, 0,
Inf*ones(1,7*length(R))];
    params_opt = optimize(params_init, objectPoints, imagePoints,
variations, options);

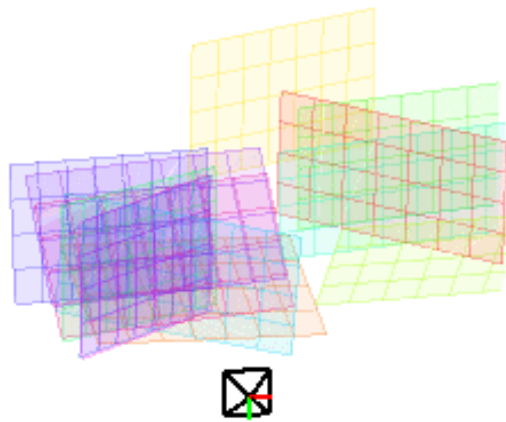
    options = optimoptions('lsqnonlin', 'Display', 'final-detailed',
'FunctionTolerance', 1e-20, 'StepTolerance', 1e-20, 'MaxIterations', 2000,
'Algorithm', 'levenberg-marquardt');

    variations = [0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, Inf, Inf, 0,
0.2*ones(1,7*length(R))];
    params_opt = optimize(params_opt, objectPoints, imagePoints, variations,
options);

    options = optimoptions('lsqnonlin', 'Display', 'final-detailed',
'FunctionTolerance', 1e-30, 'StepTolerance', 1e-20, 'MaxIterations', 2000,
'Algorithm', 'levenberg-marquardt');

    variations = [0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, Inf,
0.2*ones(1,7*length(R))];
    params_opt = optimize(params_opt, objectPoints, imagePoints, variations,
options);
end
```

```
function params_opt = optimize(params_init, objectPoints, imagePoints,  
variations, options)  
  
params_lower = params_init .* (1 - variations);  
params_upper = params_init .* (1 + variations);  
  
idx = params_init < 0;  
tmp = params_upper(idx);  
params_upper(idx) = params_lower(idx);  
params_lower(idx) = tmp;  
  
idx = params_init == 0;  
params_upper(idx) = variations(idx);  
params_lower(idx) = -variations(idx);  
  
params_opt = lsqnonlin(@(params) reprojectionError(params, objectPoints,  
imagePoints), params_init, params_lower, params_upper, options);
```



By combining the linear solution for initial parameter estimation and the non-linear optimization for refinement, Zhang's method provides a robust and accurate way to calibrate cameras, which is crucial for precise measurements and accurate spatial understanding in computer vision tasks.

♥ Like 5

🔖 **Tags:** calibration dlt homography matlab robotics

📁 **Categories:** Vision

📅 **Updated:** July 27, 2025